BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN LẬP TRÌNH MẠNG

CODEJUDGE

HỆ THỐNG CHẤM BÀI LẬP TRÌNH C++ PHÂN TÁN

Giảng viên hướng dẫn: Ngô Văn Quyền

Số nhóm: 08

Thành viên:

B23DCCN259 Nguyễn Trường Giang

B23DCCN609 Đặng Thảo Nguyên

B23DCCN301 Lê Nguyên Hiếu

B23DCCN828 Đàm Thuận Toàn

Hà Nội, năm 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN LẬP TRÌNH MẠNG

CODEJUDGE

HỆ THỐNG CHẤM BÀI LẬP TRÌNH C++ PHÂN TÁN

Giảng viên hướng dẫn: Ngô Văn Quyền

Số nhóm: 08

Thành viên:

B23DCCN259 Nguyễn Trường Giang

B23DCCN609 Đặng Thảo Nguyên

B23DCCN301 Lê Nguyên Hiếu

B23DCCN828 Đàm Thuận Toàn

Hà Nội, năm 2026

Nhận xét của giảng viên chấm bài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: .....

Điểm chữ: .....

Giảng viên (ký và ghi rõ họ tên): .....

LỜI CAM ĐOAN

Chúng em xin cam đoan báo cáo bài tập lớn với đề tài "CodeJudge – Hệ thống chấm bài lập trình C++ phân tán" là kết quả của quá trình tìm hiểu, phân tích, thiết kế và triển khai do các thành viên trong nhóm thực hiện dưới sự hướng dẫn của giảng viên.

Các nội dung trình bày trong báo cáo được xây dựng trên cơ sở kết quả thực hiện dự án và các tài liệu tham khảo có liên quan. Những thông tin, số liệu và kết quả thực nghiệm được trình bày một cách trung thực, khách quan. Các tài liệu, công trình nghiên cứu, mã nguồn và thư viện của bên thứ ba được tham khảo, sử dụng và trích dẫn nguồn theo quy định.

Chúng em xin chịu trách nhiệm về tính trung thực, chính xác và nguồn gốc của các nội dung được trình bày trong báo cáo.

ĐẠI DIỆN NHÓM

(Ký và ghi rõ họ tên)

Nguyễn Trường Giang

TÓM TẮT

CodeJudge là hệ thống chấm bài lập trình C++ phân tán được xây dựng trong học phần Lập trình mạng. Trọng tâm của dự án là tự thiết kế luồng trao đổi và điều phối công việc giữa Master Spring Boot và các Worker Python. Khi người dùng nộp bài, Master lưu trạng thái vào PostgreSQL, đưa bài vào Redis Streams theo mức ưu tiên, rồi giao cho Worker đang rảnh qua kết nối TCP. Giao thức sử dụng khung tin gồm magic byte, opcode, độ dài payload và nội dung JSON. Worker biên dịch, chạy các testcase trong môi trường go-judge và gửi tiến trình cùng kết quả chấm về Master; WebSocket chuyển các cập nhật này tới giao diện React.

Hệ thống hiện thực heartbeat để phát hiện Worker mất kết nối, giao lại bài đang xử lý và dùng attemptToken để loại bỏ kết quả cũ sau khi giao lại. Bên cạnh chức năng chấm bài, dự án còn hỗ trợ quản lý lớp học, học phần, kỳ thi, bảng xếp hạng, giám sát thi và phát hiện mã nguồn tương đồng.

Từ khóa: Online Judge; TCP Socket; Redis Streams; Master–Worker; WebSocket; Fault Tolerance.

MỤC LỤC

LỜI CAM ĐOAN **4**

TÓM TẮT **5**

MỤC LỤC **6**

DANH MỤC HÌNH **8**

DANH MỤC BẢNG **8**

DANH MỤC VIẾT TẮT **9**

CHƯƠNG 1. GIỚI THIỆU **1**

1.1. Bối cảnh và lý do thực hiện **1**

1.2. Mục tiêu và phạm vi báo cáo **2**

CHƯƠNG 2. CÁC HỆ THỐNG LIÊN QUAN VÀ THÁCH THỨC **3**

2.1. Những hệ thống liên quan **3**

2.1.1. Codeforces 3

2.1.2. Judge0 3

2.1.3. DOMjudge 4

2.2. Những thách thức còn lại trong phạm vi dự án **4**

2.3. Tổng hợp lựa chọn và quyết định thiết kế **5**

CHƯƠNG 3. BÀI TOÁN VÀ HƯỚNG TIẾP CẬN **7**

3.1. Phát biểu bài toán **7**

3.2. Use Case hệ thống **8**

CHƯƠNG 4. PHƯƠNG PHÁP & KIẾN TRÚC HỆ THỐNG **9**

4.1. Phương pháp tiếp cận **9**

4.2. Kiến trúc tổng thể **9**

4.3. Các cơ chế kỹ thuật trọng tâm **10**

CHƯƠNG 5. THIẾT KẾ GIAO TIẾP MẠNG VÀ XỬ LÝ LỖI 11

5.1. Giao thức TCP giữa Master và Worker 11

5.2. REST, WebSocket và vòng đời Submission 12

5.3. Đồng thời, Heartbeat và Failover 13

CHƯƠNG 6. HIỆN THỰC CÁC THÀNH PHẦN KỸ THUẬT 15

6.1. Master, hàng đợi và lựa chọn Worker 15

6.2. Worker Python và sandbox chạy chương trình 16

CHƯƠNG 7. THIẾT LẬP THỰC NGHIỆM 17

7.1. Môi trường và phương pháp kiểm thử 17

7.2. Các kịch bản kiểm thử và kết quả 17

7.2.1. Kiểm tra tính chính xác của kết quả chấm 17

7.2.2. Kiểm thử khả năng tiếp nhận tải 18

7.2.3. Đánh giá khả năng mở rộng Worker 20

7.2.4. Kiểm thử chịu lỗi và phục hồi 21

7.3. Đánh giá và thảo luận kết quả 22

CHƯƠNG 8. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN 23

8.1. Hạn chế của hệ thống 23

8.2. Hướng phát triển 23

KẾT LUẬN 24

TÀI LIỆU THAM KHẢO 25

DANH MỤC HÌNH

Hình 1: Use Case tổng quan hệ thống 8
Hình 2: Kiến trúc tổng thể và các giao thức kết nối trong CodeJudge 10
Hình 3: Sáu verdict khớp kỳ vọng; biểu đồ từ CSV thực nghiệm 18
Hình 4: Phân bố kết quả 1.000 POST của k6, tổng hợp từ dữ liệu gốc 19
Hình 5: Thông lượng, thời gian chờ và verdict p95 theo số Worker 20
Hình 6: Trình tự sự kiện khi ngắt Worker và khi khởi động lại Master.....21

DANH MỤC BẢNG

Bảng 1: Các loại bản tin của giao thức TCP Master–Worker.....11
Bảng 2: Kết quả đối chiếu sáu verdict sau khi sửa lỗi phân loại.....18
Bảng 3: . Tiếp nhận bài nộp trong ba lượt k6 (trạng thái cuối cửa sổ quan sát). 19
Bảng 4: Kết quả trung bình của ba lượt đo cho mỗi cấu hình Worker 20
Bảng 5: Kết quả hai phép thử phục hồi trong cửa sổ quan sát ban đầu 21

DANH MỤC VIẾT TẮT

STT	Từ viết tắt	Tên đầy đủ / Giải thích
1	TCP	Transmission Control Protocol
2	HTTP	Hypertext Transfer Protocol
3	REST	Representational State Transfer
4	WebSocket	Kênh giao tiếp hai chiều
5	FIFO	First In, First Out
6	AC	Accepted
7	WA	Wrong Answer
8	TLE	Time Limit Exceeded
9	MLE	Memory Limit Exceeded
10	RTE	Runtime Error
11	CE	Compilation Error
12	SEC	Security Error
13	VU	Virtual User
14	PEL	Pending Entries List

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và lý do thực hiện

Trong môi trường học tập và thi lập trình, các hệ thống chấm bài tự động (Online Judge) đóng vai trò quan trọng trong việc đánh giá bài làm của sinh viên. Thông qua hệ thống, sinh viên có thể nộp mã nguồn và nhận kết quả chấm dựa trên các bộ dữ liệu kiểm thử (test case) được xây dựng trước. Tuy nhiên, trong các kỳ thi lập trình có số lượng lớn thí sinh, nhiều bài nộp có thể được gửi đến hệ thống trong cùng một khoảng thời gian ngắn, tạo ra áp lực đáng kể lên quá trình tiếp nhận và xử lý yêu cầu. Mỗi bài nộp cần trải qua nhiều bước, bao gồm kiểm tra tính hợp lệ, lưu trữ thông tin, biên dịch mã nguồn, thực thi các bộ kiểm thử và trả kết quả cho người dùng.

Đối với phương pháp chấm bài tuần tự, các bài nộp được xử lý lần lượt theo thứ tự tiếp nhận. Cách tiếp cận này giúp đơn giản hóa việc quản lý trạng thái, hạn chế xung đột tài nguyên và bảo đảm mỗi công việc được xử lý theo một trình tự rõ ràng. Tuy nhiên, khi số lượng bài nộp tăng cao, những bài đến sau phải chờ các bài trước hoàn thành, ngay cả khi hệ thống vẫn còn tài nguyên tính toán chưa được sử dụng. Đặc biệt, do thời gian biên dịch và thực thi của mỗi chương trình không giống nhau, một bài nộp mất nhiều thời gian xử lý có thể làm gia tăng thời gian chờ của toàn bộ hàng đợi, dẫn đến độ trễ lớn và làm giảm khả năng đáp ứng của hệ thống.

Ngược lại, phương pháp chấm bài đồng thời cho phép nhiều bài nộp được xử lý song song, qua đó tận dụng tốt hơn tài nguyên và giảm thời gian chờ trong điều kiện phù hợp. Tuy nhiên, cách tiếp cận này cũng làm phát sinh những thách thức về điều phối công việc, đồng bộ trạng thái và quản lý kết quả. Khi nhiều tiến trình hoặc máy chấm cùng hoạt động, hệ thống cần bảo đảm một bài nộp không bị xử lý trùng lặp ngoài ý muốn, không xảy ra xung đột khi cập nhật dữ liệu và không làm thất lạc công việc trong quá trình phân phối. Ngoài ra, các tình huống như máy chấm mất kết nối sau khi nhận nhiệm vụ, quá trình xử lý bị gián đoạn hoặc kết quả của lần chấm trước được

trả về sau khi bài đã được giao lại cũng có thể ảnh hưởng đến tính chính xác và nhất quán của hệ thống.

Từ những vấn đề trên, việc xây dựng một hệ thống chấm bài tự động không chỉ dừng lại ở khả năng tiếp nhận mã nguồn và trả kết quả, mà còn đòi hỏi một cơ chế giao tiếp mạng, điều phối và xử lý đồng thời phù hợp. Hệ thống cần cân bằng giữa khả năng đáp ứng, hiệu quả sử dụng tài nguyên và độ tin cậy trong quá trình chấm bài, đặc biệt khi số lượng yêu cầu tăng đột biến hoặc một thành phần xử lý gặp sự cố. Đây chính là bài toán đặt ra đối với việc thiết kế và triển khai hệ thống chấm bài lập trình phân tán.

1.2. Mục tiêu và phạm vi báo cáo

CodeJudge được xây dựng nhằm vận dụng các kiến thức lập trình mạng vào việc thiết kế và triển khai một hệ thống chấm bài lập trình phân tán. Trong đó, Master đóng vai trò máy chủ điều phối, duy trì TCP Server để tiếp nhận và quản lý các kết nối dài hạn từ Worker. Dữ liệu trao đổi giữa các thành phần được tổ chức thành các thông điệp có cấu trúc và đóng khung trước khi truyền qua mạng. Hệ thống sử dụng Message Queue để tách biệt quá trình tiếp nhận và xử lý bài nộp, đồng thời áp dụng cơ chế heartbeat để theo dõi trạng thái kết nối và phát hiện các Worker gặp sự cố. Báo cáo tập trung phân tích những lựa chọn kỹ thuật, cách thức giao tiếp và phối hợp giữa các thành phần, cũng như phương pháp kiểm thử nhằm đánh giá tính chính xác, khả năng xử lý đồng thời và khả năng chịu lỗi của hệ thống. Bên cạnh đó, giao diện được thiết kế theo mô hình hệ thống quản lý học tập trực tuyến (LMS), giúp đưa các chức năng chấm bài vào một môi trường ứng dụng thực tế.

Mục tiêu của đề tài không phải là chứng minh CodeJudge có hiệu năng vượt trội so với các nền tảng chấm bài hiện có hay đề xuất một thuật toán hoàn toàn mới. Thay vào đó, nhóm hướng tới xây dựng một hệ thống có kiến trúc phân tán, trong đó các cơ chế giao tiếp mạng, điều phối công việc và xử lý sự cố được thiết kế, triển khai và đánh giá dựa trên những kiến thức đã học trong học phần Lập trình mạng.

CHƯƠNG 2. CÁC HỆ THỐNG LIÊN QUAN VÀ THÁCH THỨC

2.1. Những hệ thống liên quan

2.1.1. Codeforces

Codeforces là nền tảng thi đấu lập trình trực tuyến, cho phép nhiều thí sinh tham gia các cuộc thi, nộp mã nguồn và nhận kết quả chấm tự động. Hệ thống cung cấp kho bài tập, tổ chức các cuộc thi định kỳ và cập nhật bảng xếp hạng dựa trên kết quả của người tham gia. Codeforces là một ví dụ tiêu biểu về ứng dụng Online Judge trong môi trường có lượng bài nộp lớn, đặc biệt tại những thời điểm diễn ra cuộc thi (Codeforces, n.d.).

Tuy nhiên, Codeforces hướng đến cộng đồng lập trình rộng lớn, trong khi CodeJudge được xây dựng nhằm phục vụ hoạt động học tập và thi lập trình trong phạm vi nội bộ Học viện. Vì vậy, nhóm không tập trung phát triển một nền tảng có quy mô tương đương mà ưu tiên đáp ứng nhu cầu sử dụng thực tế với chi phí phần cứng phù hợp.

2.1.2. Judge0

Došilović và Mekterović (2020) giới thiệu Judge0 như một hệ thống thực thi mã nguồn trực tuyến có kiến trúc mô-đun, hỗ trợ biên dịch và chạy chương trình trong môi trường cách ly. Nền tảng cung cấp HTTP API, cho phép các ứng dụng khác tích hợp chức năng chấm bài mà không cần tự xây dựng toàn bộ cơ chế thực thi mã nguồn. Judge0 cũng hỗ trợ triển khai phân tán nhằm đáp ứng nhu cầu xử lý của nhiều ứng dụng khác nhau.

Tuy nhiên, việc sử dụng trực tiếp API của Judge0 sẽ không thể hiện rõ quá trình tự thiết kế giao thức và điều phối công việc giữa các máy chấm. Vì vậy, CodeJudge lựa chọn tự triển khai cơ chế giao tiếp Master–Worker và quản lý hàng đợi, nhằm làm rõ những vấn đề kỹ thuật thuộc phạm vi môn Lập trình mạng.

2.1.3. DOMjudge

DOMjudge là hệ thống chấm bài tự động mã nguồn mở, được sử dụng trong các cuộc thi lập trình theo mô hình ICPC. Hệ thống hỗ trợ tiếp nhận bài nộp, biên dịch, thực thi chương trình, đánh giá kết quả và quản lý bảng xếp hạng. Đặc biệt, DOMjudge sử dụng kiến trúc nhiều judgehost để phân phối công việc chấm bài, cho thấy mô hình xử lý phân tán đã được áp dụng trong thực tế (DOMjudge, n.d.).

Kế thừa định hướng xử lý phân tán, CodeJudge tập trung xây dựng kiến trúc Master–Multi-Worker với phạm vi phù hợp cho hoạt động nội bộ Học viện. Điểm nhóm chú trọng không phải tạo ra một mô hình chấm bài hoàn toàn mới, mà là chủ động thiết kế giao tiếp TCP, điều phối bài nộp và xử lý sự cố để đáp ứng mục tiêu tiếp nhận 1.000 submission đồng thời, đồng thời tối ưu việc sử dụng tài nguyên phần cứng.

2.2. Những thách thức còn lại trong phạm vi dự án

Những hệ thống đi trước đã giải quyết hiệu quả bài toán thực thi chương trình, triển khai nhiều nút xử lý và tổ chức các cuộc thi lập trình. Tuy nhiên, CodeJudge được xây dựng với mục tiêu phục vụ hoạt động học tập và thi lập trình trong phạm vi nội bộ Học viện, hướng tới khả năng tiếp nhận 1.000 submission đồng thời và tối ưu chi phí sử dụng tài nguyên phần cứng. Thách thức đặt ra là xây dựng một hệ thống chấm bài phân tán có khả năng điều phối công việc hiệu quả, quản lý chính xác trạng thái bài nộp và duy trì hoạt động ổn định khi một thành phần gặp sự cố.

Có ba thách thức kỹ thuật ảnh hưởng trực tiếp đến thiết kế hệ thống. Thứ nhất, TCP hoạt động theo mô hình byte stream nên cần có cơ chế xác định ranh giới giữa các thông điệp, kể cả khi dữ liệu được nhận qua nhiều lần đọc. Thứ hai, việc sử dụng nhiều Worker làm phát sinh vấn đề xử lý đồng thời và đồng bộ trạng thái, đòi hỏi hệ thống phải tránh giao việc cho Worker đang bận hoặc để nhiều kết quả cùng cập nhật một bài nộp không đúng cách. Thứ ba, khi Worker mất kết nối hoặc quá trình chấm bị gián đoạn, Master cần xác định trạng thái công việc để có phương án xử lý phù hợp, hạn chế tình trạng bài nộp bị bỏ dở, thất lạc hoặc ghi nhận kết quả không hợp lệ.

Vì vậy, CodeJudge không hướng tới việc thay thế hay chứng minh hiệu năng vượt trội so với các nền tảng hiện có, mà tập trung giải quyết những thách thức trên trong phạm vi một hệ thống phục vụ nội bộ. Nhóm lựa chọn hướng tiếp cận phân tán Master–Multi-Worker, kết hợp cơ chế giao tiếp mạng và điều phối công việc nhằm cân bằng giữa khả năng đáp ứng, độ tin cậy và hiệu quả sử dụng tài nguyên. Các quyết định thiết kế cụ thể sẽ được trình bày trong mục tiếp theo.

2.3. Tổng hợp lựa chọn và quyết định thiết kế

Từ những thách thức đã xác định, nhóm đặt ra một số câu hỏi kỹ thuật làm cơ sở lựa chọn kiến trúc và phương pháp triển khai CodeJudge. Các quyết định được đưa ra nhằm cân bằng giữa khả năng tiếp nhận bài nộp, hiệu quả sử dụng tài nguyên và độ tin cậy của hệ thống.

Làm thế nào để tiếp nhận lượng lớn submission và điều tiết số bài chấm theo tài nguyên hiện có?

Nhóm lựa chọn tách biệt quá trình tiếp nhận và xử lý bài nộp thông qua Message Queue. Sau khi submission được lưu, công việc được đưa vào hàng đợi Redis Streams để Master phân phối cho các Worker đang sẵn sàng. Cách tiếp cận này cho phép điều tiết số lượng bài chấm thực thi theo năng lực phần cứng, thay vì phải xử lý tất cả yêu cầu ngay tại thời điểm tiếp nhận.

Làm thế nào để phân phối bài nộp giữa nhiều Worker?

Nhóm sử dụng kiến trúc Master–Multi-Worker, trong đó Master quản lý trạng thái kết nối và điều phối công việc đến các Worker. Việc lựa chọn Worker dựa trên trạng thái sẵn sàng và thời gian phản hồi mạng (RTT) đo qua heartbeat, ưu tiên Worker đang rảnh có RTT thấp nhất. Cơ chế này giúp hạn chế giao việc cho Worker đang bận và hỗ trợ phân phối công việc giữa các nút chấm.

Làm thế nào để xác định đúng ranh giới thông điệp khi Master và Worker giao tiếp?

Hệ thống sử dụng TCP Socket với giao thức đóng khung thông điệp được thiết kế riêng. Mỗi thông điệp gồm phần header chứa magic byte, opcode và độ dài payload, theo sau là dữ liệu JSON. Hai phía đọc đủ số byte theo độ dài đã xác định trước khi xử lý nội dung, giúp phân biệt chính xác các thông điệp trên dòng byte TCP.

Làm thế nào để xử lý bài nộp khi Worker mất kết nối hoặc gặp sự cố?

Nhóm kết hợp cơ chế heartbeat để theo dõi kết nối, failover để đưa bài đang chấm trở lại hàng đợi và giao lại nếu chưa vượt quá ba lần giao, đồng thời sử dụng attemptToken để phân biệt kết quả giữa các lần giao bài. Những cơ chế này nhằm hạn chế tình trạng bài nộp bị bỏ dở hoặc kết quả cũ ghi đè lên kết quả hợp lệ.

Các lựa chọn trên hình thành cơ sở thiết kế hệ thống CodeJudge. Kiến trúc tổng thể, giao thức truyền thông và cách hiện thực các cơ chế này sẽ được phân tích trong những chương tiếp theo.

CHƯƠNG 3. BÀI TOÁN VÀ HƯỚNG TIẾP CẬN

3.1. Phát biểu bài toán

Xét một hệ thống chấm bài lập trình trực tuyến, trong đó nhiều sinh viên có thể gửi mã nguồn C++17 thông qua trình duyệt. Mỗi bài nộp tương ứng với một bài toán có bộ dữ liệu kiểm thử được lưu trong cơ sở dữ liệu. Sau khi yêu cầu được xác thực và kiểm tra hợp lệ, hệ thống ghi nhận bài nộp, đưa công việc vào hàng đợi và phân phối đến Worker đang sẵn sàng. Worker tiếp nhận nhiệm vụ, sử dụng môi trường sandbox để biên dịch và lần lượt thực thi các bộ kiểm thử trước khi gửi kết quả về Master. Trong quá trình này, trạng thái bài nộp được cập nhật đến trình duyệt thông qua WebSocket mà không cần duy trì yêu cầu HTTP cho đến khi chấm hoàn tất.

Khi Worker mất kết nối hoặc quá trình chấm bị gián đoạn, Master thực hiện xử lý và giao lại công việc trong giới hạn số lần cho phép. Đồng thời, cơ chế attemptToken được sử dụng để phân biệt các lần giao bài, ngăn kết quả từ lần xử lý cũ ghi đè lên kết quả hợp lệ.

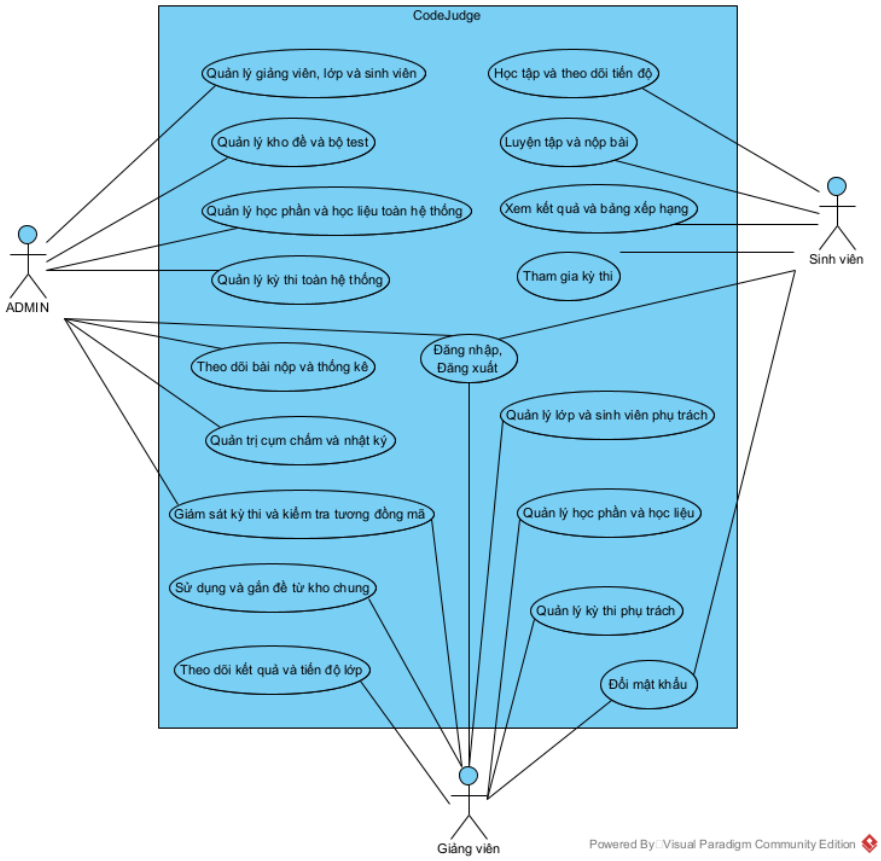
Đầu vào của quá trình chấm bao gồm bài toán được lựa chọn, mã nguồn C++17 và các giới hạn thời gian, bộ nhớ được cấu hình theo đề bài. Đầu ra bao gồm trạng thái xử lý, điểm số và kết quả của các bộ kiểm thử đã thực thi. Các nhãn kết quả hiện được Worker tạo ra gồm AC (Accepted), WA (Wrong Answer), TLE (Time Limit Exceeded), MLE (Memory Limit Exceeded), RTE (Runtime Error) và CE (Compilation Error).

Để thực hiện quá trình trên, CodeJudge sử dụng PostgreSQL để lưu trữ thông tin và trạng thái bài nộp, Redis Streams để quản lý hàng đợi công việc, WebSocket để cập nhật trạng thái đến trình duyệt và kết nối TCP dài hạn giữa Master với Worker để truyền nhận nhiệm vụ chấm.

3.2. Use Case hệ thống

Ở góc độ chức năng, CodeJudge xác định ba nhóm tác nhân chính gồm Sinh viên, Giảng viên và Quản trị viên. Mỗi nhóm có phạm vi sử dụng và quyền hạn khác nhau. Sinh viên tham gia học tập, luyện tập lập trình, nộp bài, tham gia kỳ thi và theo dõi kết quả. Giảng viên quản lý lớp học, học phần, bài tập và kỳ thi trong phạm vi được phân công, đồng thời theo dõi tiến độ học tập và kết quả của sinh viên. Quản trị viên chịu trách nhiệm quản lý người dùng, lớp học, kho đề bài, kỳ thi và giám sát hoạt động của hệ thống chấm bài.

Các chức năng chính và mối quan hệ giữa tác nhân với hệ thống được thể hiện trong Sơ đồ Use Case tổng quan dưới đây. Sơ đồ nhằm khái quát phạm vi chức năng của CodeJudge, trong khi báo cáo tập trung phân tích kiến trúc phân tán, cơ chế giao tiếp mạng và quá trình điều phối công việc thay vì đi sâu vào đặc tả từng Use Case.



Hình 1: Use Case tổng quan hệ thống

CHƯƠNG 4. PHƯƠNG PHÁP & KIẾN TRÚC HỆ THỐNG

4.1. Phương pháp tiếp cận

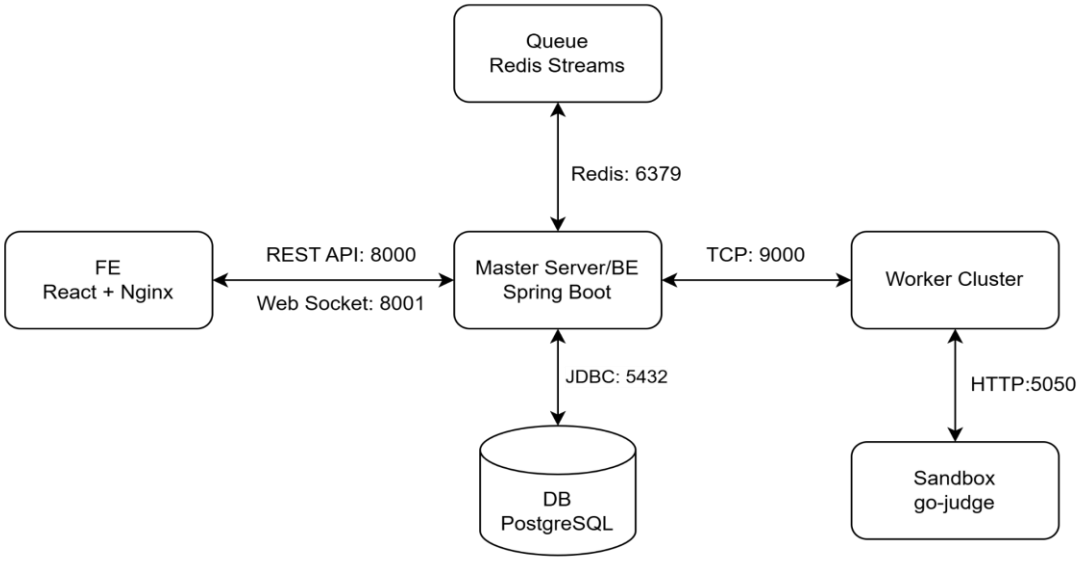
CodeJudge được xây dựng theo kiến trúc phân tán Master–Multi-Worker, trong đó quá trình tiếp nhận yêu cầu, điều phối công việc và thực thi mã nguồn được tách thành các thành phần có trách nhiệm riêng. Thay vì chấm bài trực tiếp trong một yêu cầu HTTP, hệ thống sử dụng Message Queue để lưu trữ công việc và phân phối đến các Worker đang sẵn sàng.

Phương pháp này hướng tới khả năng tiếp nhận số lượng lớn bài nộp trong thời gian ngắn, đồng thời điều tiết số lượng tác vụ chấm theo năng lực phần cứng. Nhóm tập trung tự triển khai giao thức truyền thông TCP, cơ chế điều phối, quản lý trạng thái và xử lý sự cố; riêng môi trường thực thi mã nguồn được kế thừa từ go-judge.

4.2. Kiến trúc tổng thể

Kiến trúc CodeJudge gồm các thành phần chính: Frontend (React và Nginx), Backend/Master (Spring Boot), Redis Streams, PostgreSQL, Python Worker và go-judge. Master đóng vai trò trung tâm, chịu trách nhiệm xử lý yêu cầu, quản lý hàng đợi và điều phối công việc. Các Worker duy trì kết nối TCP dài hạn với Master, nhận nhiệm vụ và sử dụng go-judge để biên dịch, thực thi chương trình trong môi trường sandbox.

Redis Streams đảm nhận lưu trữ hàng đợi công việc, trong khi PostgreSQL lưu dữ liệu và trạng thái xử lý. Các thành phần được kết nối qua REST API, WebSocket, TCP và HTTP, tạo thành một hệ thống có thể theo dõi và điều chỉnh số lượng Worker theo nhu cầu.



Hình 2: Kiến trúc tổng thể và các giao thức kết nối trong CodeJudge

4.3. Các cơ chế kỹ thuật trọng tâm

Custom Application-Layer Protocol: Master và Worker giao tiếp bằng TCP Socket với giao thức đóng khung thông điệp tự định nghĩa, gồm magic byte, opcode, độ dài payload và dữ liệu JSON. Cơ chế này giúp xác định ranh giới thông điệp trên dòng byte TCP.

Producer–Consumer Queue: Redis Streams được sử dụng làm hàng đợi trung gian, tách biệt quá trình tiếp nhận và thực thi bài chấm. Các Worker có thể xử lý công việc đồng thời, trong khi Master quản lý việc phân phối theo trạng thái sẵn sàng và năng lực xử lý.

Automatic Failover: Master sử dụng heartbeat để phát hiện Worker mất kết nối. Công việc đang xử lý được đưa trở lại hàng đợi nếu chưa vượt quá giới hạn ba lần giao. Cơ chế attemptToken giúp loại bỏ kết quả thuộc những lần giao cũ.

Priority-aware Latency-based Task Scheduling: Hệ thống sử dụng các hàng đợi ưu tiên cho từng loại công việc. Khi điều phối, Master lựa chọn Worker đang rảnh có thời gian phản hồi mạng (RTT) thấp nhất, nhằm phân phối nhiệm vụ theo trạng thái kết nối và chính sách ưu tiên.

CHƯƠNG 5. THIẾT KẾ GIAO TIẾP MẠNG VÀ XỬ LÝ LỖI

5.1. Giao thức TCP giữa Master và Worker

CodeJudge sử dụng giao thức TCP tự định nghĩa để trao đổi dữ liệu giữa Master (Java) và Worker (Python). Master mở TCP Server tại cổng 9000, trong khi Worker chủ động kết nối và duy trì phiên làm việc để nhận nhiệm vụ chấm. Thay vì truyền JSON trực tiếp, mỗi thông điệp được đóng khung theo một cấu trúc thống nhất, giúp hai thành phần xác định chính xác ranh giới dữ liệu trên dòng byte TCP.

Mỗi bản tin gồm Header cố định 6 byte và phần Payload chứa dữ liệu JSON UTF-8, được tổ chức như sau:

[Magic: 1 byte] [Opcode: 1 byte] [Payload Length: 4 byte] [JSON Payload]

Trong đó, Magic 0xAA là giá trị nhận dạng bản tin hợp lệ, Opcode xác định loại thông điệp và Payload Length cho biết số byte dữ liệu cần đọc, được mã hóa theo thứ tự big-endian. Phía Java sử dụng DataInputStream và readFully() để đọc đủ dữ liệu, đồng thời giới hạn Payload tối đa 10 × 1024 × 1024 byte. Phía Python sử dụng struct.pack(">I") khi gửi và _recv_exact() khi nhận, bảo đảm tương thích với định dạng bản tin của Master.

Bảng 1: Các loại bản tin của giao thức TCP Master–Worker

Opcode	Bản tin	Chức năng
0x01	HEARTBEAT	Kiểm tra kết nối và phản hồi trạng thái Worker
0x02	WORKER_REGISTER	Worker đăng ký với Master

0x03	TASK_ASSIGN	Master giao nhiệm vụ chấm và dữ liệu kiểm thử
0x04	TASK_STATUS	Cập nhật trạng thái COMPILING, TESTING
0x05	TASK_RESULT	Trả kết quả chấm, điểm số và attemptToken

Cách tổ chức này cho phép Master giao bài, theo dõi tiến trình và nhận kết quả thông qua một kết nối TCP dài hạn.

5.2. REST, WebSocket và vòng đời Submission

CodeJudge kết hợp REST API và WebSocket để tách biệt quá trình tiếp nhận bài nộp với quá trình trả kết quả chấm. Khi sinh viên gửi mã nguồn, trình duyệt gọi POST /api/submissions với các thông tin như problemId, sourceCode, language và contestId nếu có. Master tạo bản ghi submission và trả về đối tượng chứa id mà không cần chờ quá trình chấm hoàn thành.

Sau khi được tiếp nhận, bài nộp được đưa vào Redis Streams và chuyển đến Worker khi có tài nguyên xử lý. Trong quá trình chấm, Worker gửi các trạng thái COMPILING, TESTING và kết quả cuối cùng về Master. Các thay đổi được cập nhật vào hệ thống và thông báo tới trình duyệt qua WebSocket. Nhờ đó, sinh viên có thể theo dõi tiến trình chấm mà không phải duy trì yêu cầu HTTP ban đầu.

WebSocket sử dụng các bản tin JSON, trong đó client gửi AUTH khi thiết lập kết nối. Server phát các sự kiện như SUBMISSION_UPDATE, WORKER_UPDATE và LOG. Tuy nhiên, mã nguồn hiện sử dụng broadcastText() để gửi các sự kiện này tới mọi kết nối WebSocket đang mở, chưa giới hạn người nhận dựa trên trạng thái xác

thực. Đây là vấn đề cần hoàn thiện nếu hệ thống được triển khai trong môi trường có yêu cầu phân quyền và bảo mật dữ liệu.

Khi WebSocket bị gián đoạn, trình duyệt có thể bỏ lỡ một số thông báo mặc dù quá trình chấm vẫn tiếp tục. REST API truy vấn chi tiết submission cho phép lấy lại trạng thái hiện tại, nhưng frontend hiện chưa tự động thực hiện truy vấn này ngay sau khi WebSocket kết nối lại.

Về quản lý dữ liệu, vòng đời bài nộp được phản ánh qua ba thành phần: PostgreSQL lưu trạng thái và kết quả chấm, Redis Streams quản lý công việc đang chờ xử lý, còn WorkerSession thể hiện tình trạng sẵn sàng của từng Worker. Việc phối hợp ba thành phần giúp Master xác định bài nào đang chờ, bài nào đã được giao và kết quả nào cần được ghi nhận.

5.3. Đồng thời, Heartbeat và Failover

Để điều phối nhiều Worker, Master sử dụng ConcurrentHashMap quản lý các phiên kết nối và synchronized tại dispatchNext() nhằm tránh nhiều luồng cùng lựa chọn, giao công việc tại một thời điểm. Phương thức WorkerSession.send() đồng bộ thao tác ghi dữ liệu trên socket; phía Python sử dụng _write_lock để ngăn các thread ghi xen kẽ vào cùng kết nối TCP.

Worker tạo một threading.Thread cho mỗi TASK_ASSIGN và sử dụng biến _busy để phản ánh trạng thái đang xử lý. Tuy nhiên, _busy chỉ là biến trạng thái, chưa phải cơ chế giới hạn đồng thời cứng. Vì vậy, Worker cần được bổ sung cơ chế kiểm soát số lượng nhiệm vụ thực thi để tránh nhận quá khả năng xử lý trong những tình huống ngoài dự kiến.

Master kiểm tra heartbeat theo chu kỳ 5 giây và xác định Worker mất kết nối khi vượt ngưỡng 15 giây. Khi đó, bài nộp đang được xử lý có thể được đưa trở lại hàng đợi để giao cho Worker khác, với giới hạn tối đa ba lần giao. Cơ chế này giúp hạn chế tình trạng bài nộp bị bỏ dở khi một Worker gặp sự cố.

Một vấn đề quan trọng của hệ thống phân tán là kết quả chấm có thể đến muộn. Chẳng hạn, Worker đã hoàn thành công việc nhưng mất kết nối trước khi Master nhận được kết quả. Master có thể giao lại bài cho Worker khác, trong khi kết quả của lần chấm trước vẫn được gửi về sau đó. Để xử lý tình huống này, mỗi lần giao bài được gắn một attemptToken riêng. Khi nhận TASK_RESULT, Master đối chiếu token với lần giao hiện tại và bỏ qua kết quả không còn hợp lệ.

Cơ chế trên không bảo đảm một bài nộp chỉ được thực thi đúng một lần (*exactly-once execution*), bởi cùng một mã nguồn vẫn có thể được chạy lại sau sự cố. Tuy nhiên, việc kiểm tra attemptToken giúp hạn chế nguy cơ kết quả cũ ghi đè lên kết quả của lần chấm hợp lệ.

Đối với Redis Streams, Master sử dụng XACK để xác nhận bản tin đã được xử lý. Trong luồng nhận kết quả, thao tác này được gọi sau submissionRepository.save() nhưng trước khi transaction cơ sở dữ liệu hoàn tất. Ngoài ra, XACK còn xuất hiện trong các tình huống xử lý failover hoặc loại bỏ bản tin không còn hợp lệ. Vì vậy, chưa thể khẳng định quá trình xác nhận hàng đợi và lưu dữ liệu luôn bảo đảm tính nguyên tử khi xảy ra sự cố.

Nhìn chung, các cơ chế đồng bộ, heartbeat, failover và attemptToken giúp CodeJudge quản lý quá trình chấm trên nhiều Worker và xử lý một số tình huống lỗi kết nối. Tuy nhiên, các trường hợp mất kết nối giữa chừng, kết quả trả về muộn và sự cố giữa thao tác lưu cơ sở dữ liệu với xác nhận hàng đợi vẫn cần được kiểm chứng thông qua các kịch bản thử nghiệm cụ thể.

CHƯƠNG 6. HIỆN THỰC CÁC THÀNH PHẦN KỸ THUẬT

6.1. Master, hàng đợi và lựa chọn Worker

Mã nguồn CodeJudge được tổ chức thành các thành phần chính: backend/ sử dụng Java 21 và Spring Boot, frontend/ sử dụng React và Vite, worker/ được phát triển bằng Python, go-judge-custom/ chứa cấu hình môi trường chấm bài và tests/k6/ phục vụ kiểm thử tải. Backend tích hợp Spring Data JPA, Redis, Flyway và Java-WebSocket để triển khai các chức năng giao tiếp, lưu trữ và điều phối. Để bảo đảm khả năng tái lập thực nghiệm, cần ghi nhận phiên bản mã nguồn, các tệp lockfile và phiên bản hoặc digest Docker image sử dụng khi kiểm thử.

Tại Master, phương thức dispatchNext() chịu trách nhiệm lựa chọn và phân phối công việc. Hệ thống lọc các Worker còn kết nối, đang rảnh và ưu tiên Worker có thời gian phản hồi mạng (RTT) thấp nhất. Khi giao bài, Master tạo attemptToken, ghi nhận Worker được chọn, tăng số lần giao, tạo sự kiện ASSIGNED và đánh dấu Worker là BUSY. Tuy nhiên, trạng thái submission vẫn là IN_QUEUE cho đến khi Master nhận thông báo COMPILING từ Worker.

Thành phần RedisStreamQueueService định nghĩa ba hàng đợi judge:contest, judge:practice và judge:custom, được đọc theo thứ tự ưu tiên. Trong luồng hiện tại, bài nộp thông thường được đưa vào hàng đợi contest hoặc practice; chức năng chạy thử gửi nhiệm vụ trực tiếp đến Worker, không sử dụng stream custom. Cách tổ chức này giúp ưu tiên xử lý bài thi, đồng thời tách quá trình tiếp nhận bài nộp khỏi việc thực thi chương trình.

Khi Master khởi động lại, hệ thống sử dụng XPENDING và XCLAIM để thu hồi một số bản ghi đang chờ xác nhận trong Redis Streams, sau đó kiểm tra các submission còn chờ trong PostgreSQL để bổ sung công việc vào hàng đợi. Đây là cơ chế hỗ trợ khôi phục sau gián đoạn, nhưng khả năng phục hồi đầy đủ vẫn cần được xác minh bằng thực nghiệm.

Nhìn chung, chính sách điều phối hiện tại đơn giản và dễ kiểm soát. Tuy nhiên, RTT không phản ánh trực tiếp mức tải CPU hoặc bộ nhớ của Worker. Ngoài ra, việc ưu tiên bài thi có thể làm tăng thời gian chờ của bài luyện tập khi hàng đợi thi liên tục có công việc mới.

6.2. Worker Python và sandbox chạy chương trình

Worker được triển khai bằng Python, có nhiệm vụ nhận công việc từ Master và điều khiển quá trình chấm thông qua go-judge. Khi nhận TASK_ASSIGN, Worker gửi trạng thái COMPILING và gọi API POST /run để biên dịch mã nguồn C++17 bằng g++, với các giới hạn tài nguyên được cấu hình. Nếu biên dịch thành công, chương trình được thực thi lần lượt trên từng bộ kiểm thử thông qua môi trường sandbox.

Trong quá trình chạy test, Worker thu thập thời gian thực thi, bộ nhớ sử dụng và kết quả chương trình. Với các test thực thi thành công, hàm _check_lines() so sánh kết quả đầu ra với đáp án, bỏ qua khoảng trắng ở cuối mỗi dòng thay vì so sánh tuyệt đối từng byte. Worker tổng hợp số test đạt, điểm số và các thông tin cần thiết để xác định kết quả cuối cùng như AC, WA, TLE, MLE, RTE hoặc CE. Sau khi hoàn thành, Worker gửi TASK_RESULT kèm attemptToken về Master và yêu cầu dọn dẹp các tệp tạm. Mặc dù backend có định nghĩa nhãn SEC, Worker hiện chưa tạo kết quả này.

Về triển khai, các Worker sử dụng chung dịch vụ go-judge được cấu hình trong Docker Compose. Image tại go-judge-custom/ kế thừa từ criyle/go-judge và bổ sung công cụ biên dịch cần thiết. Việc sử dụng sandbox có sẵn giúp nhóm tập trung vào giao tiếp Master–Worker và điều phối công việc thay vì tự xây dựng môi trường thực thi cách ly.

CHƯƠNG 7. THIẾT LẬP THỰC NGHIỆM

7.1. Môi trường và phương pháp kiểm thử

Thực nghiệm được thực hiện trên môi trường CodeJudge chạy cục bộ bằng Docker Compose. Máy thử HP Pavilion 15-eg2xxx có 16 CPU logic và 16 GiB RAM; Docker Desktop được cấp 16 CPU logic và 7,59 GiB RAM. Các thành phần chính gồm PostgreSQL 18, Redis 7, Master Spring Boot, Worker Python và go-judge (parallelism = 4, giới hạn container 4 GiB). Bài A+B được dùng làm dữ liệu thử với bốn testcase, giới hạn 1.000 ms và 256 MiB cho mỗi test.

Nhóm sử dụng API và k6 để phát yêu cầu, PostgreSQL và nhật ký Master–Worker để đối chiếu vòng đời bài nộp; Grafana/Prometheus và Docker stats hỗ trợ quan sát trạng thái, tài nguyên. Mỗi lượt được gắn run_id. Thời gian phản hồi HTTP chỉ đo yêu cầu POST, còn thời gian có verdict tính từ created_at đến finished_at; thời gian chờ hàng đợi dựa trên các mốc ASSIGNED và created_at/REQUEUED. Các giá trị p50, p95 được tính trên tập mẫu của từng lần chạy. Cấu hình mã nguồn được ghi nhận tại commit 985580ffb719651ca8c5cfb7aa2bf0330e1c3cc6; phiên bản image sau sửa lỗi verdict cần lưu riêng để tái lập đầy đủ.

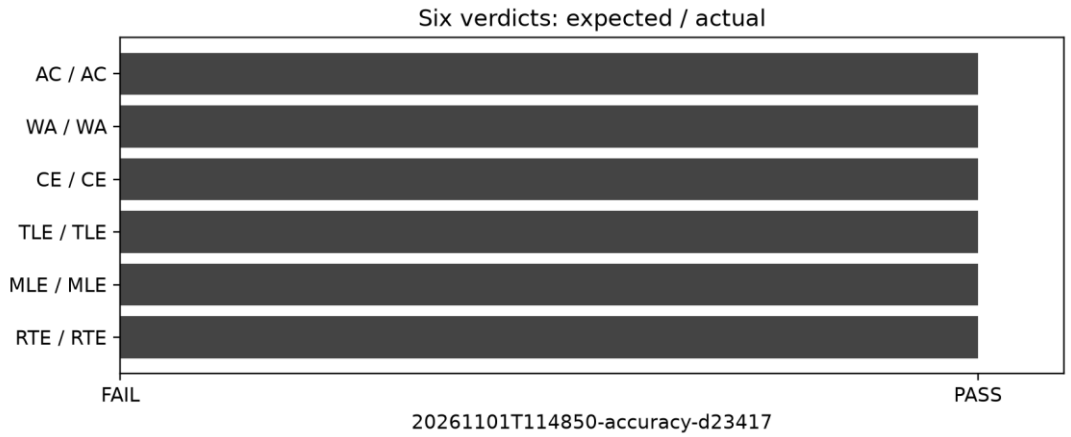
7.2. Các kịch bản kiểm thử và kết quả

7.2.1. Kiểm tra tính chính xác của kết quả chấm

Sáu chương trình C++17 được chuẩn bị để tạo ra các verdict AC, WA, CE, TLE, MLE và RTE. Kết quả được đối chiếu giữa API, bảng submission và nhật ký Worker. Ở lượt đầu, hai trường hợp TLE/MLE bị ánh xạ sai; sau khi sửa Worker và chạy lại, cả sáu trường hợp đều khớp với kỳ vọng.

Bảng 2: Kết quả đối chiếu sáu verdict sau khi sửa lỗi phân loại

Trường hợp	**AC**	**WA**	**CE**	**TLE**	**MLE**	**RTE**
Kỳ vọng	AC	WA	CE	TLE	MLE	RTE
Thực tế	AC	WA	CE	TLE	MLE	RTE
Submission ID	1014	1015	1016	1017	1018	1019
Đánh giá	Đạt	Đạt	Đạt	Đạt	Đạt	Đạt



Hình 3: Sáu verdict khớp kỳ vọng; biểu đồ từ CSV thực nghiệm

Minh chứng: run 20261101T114850-accuracy-d23417; verdicts.csv, test_results.csv, worker.log.

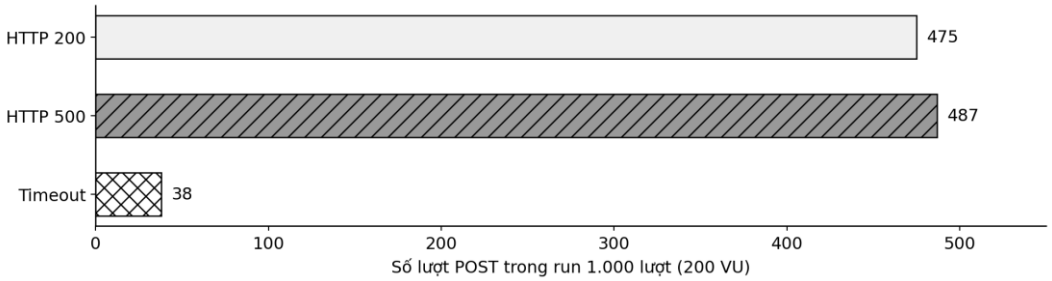
7.2.2. Kiểm thử khả năng tiếp nhận tải

k6 phát ba mức tải gồm 10, 100 và 1.000 lượt POST, với lần lượt 5, 20 và 200 virtual users (VU). Sau khi nhận submission, client tiếp tục truy vấn để theo dõi verdict. Số VU và tổng số lượt nộp không đồng nghĩa 1.000 submission xuất hiện đồng thời. Bảng sau phân biệt phản hồi HTTP, số bài thật sự ghi trong PostgreSQL và trạng thái cuối cửa sổ quan sát.

Bảng 3: . Tiếp nhận bài nộp trong ba lượt k6 (trạng thái cuối cửa sổ quan sát).

VU / POST	HTTP 200	HTTP 500	Timeout	DB ID	Hoàn tất / chưa xong
5 / 10	10	0	0	10	10 / 0
20 / 100	100	0	0	100	100 / 0
200 / 1.000	475	487	38	506	502 / 4

Ở lượt lớn, thời gian phản hồi POST p95 đạt 30.206 ms, verdict p95 đạt 91.305 ms; đỉnh số submission đang hoạt động ghi nhận là 231, thấp hơn mục tiêu 1.000. Mặc dù chỉ có 475 phản hồi HTTP 200, PostgreSQL ghi 506 ID vì một số yêu cầu đã tạo bài trước khi client nhận lỗi hoặc hết thời gian chờ. Nhật ký Master đồng thời ghi nhận HikariCP hết kết nối sẵn có (30/30 active, 176 yêu cầu chờ), cho thấy tình trạng nghẽn kết nối cơ sở dữ liệu dưới tải; chưa đủ cơ sở khẳng định đây là nguyên nhân duy nhất của tất cả lỗi.



Hình 4: Phân bố kết quả 1.000 POST của k6, tổng hợp từ dữ liệu gốc

Một lượt tải khác sử dụng bộ phát yêu cầu riêng đạt 800 HTTP 200 trên 1.000 POST, ghi nhận 863 ID và đỉnh đồng thời 858. Tại cuối cửa sổ có 862 bài hoàn tất, một bài chưa xong; bài còn lại chỉ hoàn tất sau khi Master được khởi động lại. Hai lượt dùng cách phát tải khác nhau nên không gộp để tính một chỉ số hiệu năng chung. Trong lượt này, thời gian biên dịch trung bình đo tại Worker là 358,99 ms và thời gian chạy test trung bình 60,81 ms trên tập mẫu thu được.

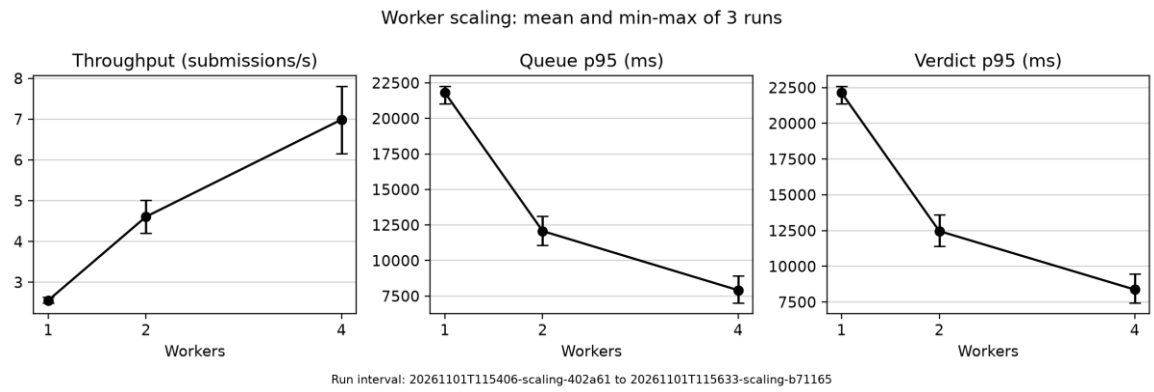
Minh chứng: run k6 20261101T120359-k6-cd1392 và load 20261101T114930-load-5973b9; k6.csv, summary.json, submissions.csv, backend.log, worker_timings.csv.

7.2.3. Đánh giá khả năng mở rộng Worker

Mỗi cấu hình 1, 2 và 4 Worker được thử ba lượt độc lập, mỗi lượt 60 submission cùng bài toán, bộ test và điều kiện phát tải (20 luồng). Số Worker kết nối được xác nhận trước khi chạy. Bảng 7.3 trình bày giá trị trung bình của ba lượt; thông lượng có khoảng dao động min–max trong dữ liệu gốc.

Bảng 4: Kết quả trung bình của ba lượt đo cho mỗi cấu hình Worker

Worker	Thông lượng (bài/s)	Queue p95 (s)	Verdict p95 (s)	BUSY (%)	Hoàn tất
1	2,56	21,81	22,14	93,88	180/180
2	4,61	12,07	12,46	94,31	180/180
4	6,99	7,91	8,37	93,45	180/180



Hình 5: Thông lượng, thời gian chờ và verdict p95 theo số Worker

Thông lượng tăng từ 2,56 lên 6,99 bài/giây (khoảng 2,73 lần) khi tăng từ 1 lên 4 Worker; queue p95 giảm từ 21,81 xuống 7,91 giây. Mức tăng không tuyến tính theo số Worker. CPU trung bình của dịch vụ go-judge dùng chung lần lượt khoảng

82,24%, 174,54% và 322,97% theo số liệu Docker stats. Kết quả phản ánh sự gia tăng sử dụng tài nguyên, nhưng chưa chứng minh go-judge là điểm nghẽn duy nhất hay xác định được mức chi phí phần cứng tối ưu.

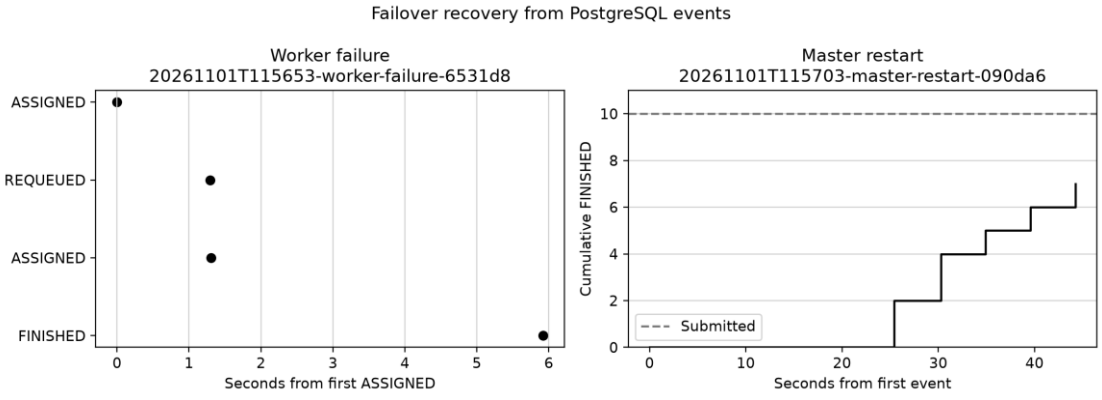
Minh chứng: 04-worker-scaling/ (9 lượt chạy); scaling.csv, resources.csv, worker_utilization.csv và log theo run_id.

7.2.4. Kiểm thử chịu lỗi và phục hồi

Nhóm thử hai tình huống: ngắt Worker đang xử lý và khởi động lại Master khi vẫn còn submission chờ hoặc đang chấm. Trạng thái được kiểm tra bằng ID bài nộp và các sự kiện ASSIGNED, REQUEUED, FINISHED trong PostgreSQL.

Bảng 5: Kết quả hai phép thử phục hồi trong cửa sổ quan sát ban đầu

Tình huống	**Kết quả quan sát**	**Đánh giá**
Ngắt Worker đang chấm	Có REQUEUED, hai ASSIGNED; kết thúc FINISHED/TLE	Đạt có giới hạn
Khởi động lại Master	7/10 bài hoàn tất; 3 bài còn TESTING sau 180 giây	Chưa đạt



Hình 6: Trình tự sự kiện khi ngắt Worker và khi khởi động lại Master

Trong phép thử ngắt Worker, sự kiện REQUEUED xuất hiện 1.295 ms sau ASSIGNED đầu tiên, việc giao lại diễn ra sau khoảng 13 ms và bài kết thúc sau tổng 5.920 ms. Đây không phải số đo heartbeat timeout vì kết nối TCP bị ngắt trực tiếp. Hai sự kiện ASSIGNED có trường attempt cùng bằng 1, trong khi submission.attempts cuối là 0; do vậy chưa đủ bằng chứng để xác nhận bộ đếm lần giao và tính duy nhất của attemptToken trong phép thử này.

Ở phép thử Master restart, ba bài còn TESTING sau 180 giây và chỉ kết thúc TLE sau một lần khởi động lại bổ sung. Vì thế kết quả của phép thử ban đầu vẫn được đánh giá là chưa đạt, không cộng việc phục hồi về sau vào thành công của lần thử.

Minh chứng: run 20261101T115653-worker-failure-6531d8 và 20261101T115703-master-restart-090da6; events_before/after.csv, summary.json, backend.log.

7.3. Đánh giá và thảo luận kết quả

Thực nghiệm đã xác nhận sáu verdict sau khi sửa lỗi ánh xạ, đồng thời cho thấy với cùng tập tải 60 bài, số Worker lớn hơn giúp tăng thông lượng và giảm thời gian chờ. Tuy nhiên, các kết quả chỉ áp dụng cho cấu hình local, bài A+B có bốn testcase và thời gian quan sát đã ghi nhận; chưa thể suy rộng sang những bộ đề có khối lượng biên dịch hoặc thực thi khác nhau.

Hai thử nghiệm 1.000 POST cho thấy hệ thống chưa đáp ứng ổn định mục tiêu 1.000 submission đồng thời. Lỗi HTTP và cảnh báo cạn pool kết nối PostgreSQL cần được xử lý, sau đó thử lại trên cùng bộ dữ liệu. Về độ tin cậy, phép thử ngắt Worker có tiến trình giao lại nhưng còn thiếu bằng chứng kiểm tra attemptToken; phép thử khởi động lại Master chưa phục hồi toàn bộ công việc trong cửa sổ ban đầu.

CHƯƠNG 8. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

8.1. Hạn chế của hệ thống

Mặc dù CodeJudge đã triển khai được kiến trúc chấm bài phân tán, hệ thống vẫn còn một số hạn chế. Kết quả thực nghiệm cho thấy việc tăng số Worker giúp cải thiện thông lượng nhưng chưa đạt hiệu quả tuyến tính do các Worker sử dụng chung dịch vụ go-judge. Khi tải lớn, hệ thống xuất hiện tình trạng cạn kết nối PostgreSQL và nhiều yêu cầu thất bại, do đó chưa đáp ứng được mục tiêu 1.000 submission đồng thời.

Bên cạnh đó, cơ chế failover chưa bảo đảm khôi phục toàn bộ công việc khi Master gặp sự cố. Việc quản lý trạng thái giữa Redis và PostgreSQL, giới hạn tài nguyên Worker và bảo mật kết nối vẫn cần hoàn thiện. Các chức năng phát hiện mã nguồn tương đồng và chống gian lận cũng còn hạn chế khi áp dụng trong những kỳ thi thực tế.

8.2. Hướng phát triển

Trong thời gian tới, nhóm hướng tới tối ưu kết nối cơ sở dữ liệu, cải thiện thuật toán điều phối và nghiên cứu triển khai sandbox độc lập để nâng cao khả năng xử lý đồng thời, đồng thời giảm chi phí phần cứng trên mỗi bài chấm.

Ngoài ra, hệ thống cần hoàn thiện cơ chế phục hồi sau sự cố, tăng cường bảo mật giao tiếp Master–Worker và mở rộng giám sát hiệu năng. Về lâu dài, nhóm định hướng cải thiện khả năng phát hiện mã nguồn sao chép, giám sát gian lận và kiểm thử trên quy mô lớn hơn để đáp ứng nhu cầu học tập, thi lập trình trong Học viện.

KẾT LUẬN

Thông qua đề tài "CodeJudge – Hệ thống chấm bài lập trình C++ phân tán", nhóm đã vận dụng các kiến thức của học phần Lập trình mạng để thiết kế và triển khai hệ thống chấm bài tự động theo kiến trúc Master–Multi-Worker. Hệ thống sử dụng giao thức TCP tự định nghĩa để trao đổi dữ liệu giữa Master và Worker, Redis Streams để quản lý hàng đợi, WebSocket để cập nhật trạng thái và go-judge để thực thi mã nguồn trong môi trường cách ly. Bên cạnh đó, nhóm đã triển khai các cơ chế điều phối theo độ ưu tiên và độ trễ mạng, heartbeat và failover nhằm hỗ trợ xử lý công việc trong môi trường phân tán.

Kết quả thực nghiệm cho thấy hệ thống phân loại chính xác sáu loại kết quả chấm sau khi khắc phục lỗi, đồng thời cải thiện thông lượng từ 2,56 lên 6,99 bài/giây khi tăng số lượng Worker từ 1 lên 4. Tuy nhiên, CodeJudge chưa đáp ứng ổn định mục tiêu 1.000 submission đồng thời và cơ chế phục hồi khi Master gặp sự cố vẫn cần được hoàn thiện.

Nhìn chung, đề tài đã đạt được mục tiêu xây dựng và kiểm chứng các thành phần cốt lõi của một hệ thống chấm bài phân tán. Quá trình thực hiện giúp nhóm hiểu rõ hơn về giao tiếp Socket, xử lý đồng thời, điều phối công việc và những thách thức về tính nhất quán, độ tin cậy trong hệ thống mạng. Đây là cơ sở để tiếp tục hoàn thiện CodeJudge và hướng tới ứng dụng trong môi trường học tập, thi lập trình tại Học viện.

TÀI LIỆU THAM KHẢO

Codeforces. (n.d.). *Codeforces*. https://codeforces.com/

criyle. (n.d.). *go-judge* [Computer software]. GitHub. https://github.com/criyle/go-judge

DOMjudge. (n.d.). *DOMjudge documentation*. https://www.domjudge.org/documentation

Došilović, H. Z., & Mekterović, I. (2020). Robust and scalable online code execution system. In *2020 43rd International Convention on Information, Communication and Electronic Technology (MIPRO)* (pp. 1627–1632). IEEE. https://doi.org/10.23919/MIPRO48935.2020.9245310

Fette, I., & Melnikov, A. (2011). *The WebSocket protocol* (RFC 6455). Internet Engineering Task Force. https://doi.org/10.17487/RFC6455

Grafana Labs. (n.d.). *Grafana k6 documentation*. https://grafana.com/docs/k6/latest/

Judge0. (n.d.). *Judge0 documentation*. https://docs.judge0.com/

Redis. (n.d.). *Redis Streams*. https://redis.io/docs/latest/develop/data-types/stream/